Traffic Sign Detection and Classification

Abstract

This project explores a two-stage traffic-sign recognition prototype. A pretrained YOLO11n detector locates selected objects, and a ResNet-18 classifier predicts a category from each crop. The five categories are stop, speed limit, traffic light, pedestrian crossing, and no entry. Two synthetic experiments are available: a small ten-image workflow run and a separate 120-image video demonstration. In the second experiment, the classifier correctly predicted 18 of 18 held-out annotated crops. However, the detector missed two categories in the demonstration and misidentified another. The 20-second input took approximately 9.91 seconds to process on CPU. These results concern generated images and do not establish real-road performance. The repository includes configuration, logging, pipeline tracking, an environment definition, and classifier export.

1. Introduction and Objective

Traffic signs communicate instructions to drivers. Recognising their location and category is a practical computer-vision task with a connection to driver assistance. This topic provides a focused alternative to recognising vehicles and predicting several attributes, while retaining the central detection and classification tasks.

The objective is to process video frames, identify candidate objects, classify their crops, and save annotated output. Detection answers where an object is. Classification answers which category a crop resembles. Separating these tasks makes their behaviour easier to inspect, but errors in detection affect the classifier's input.

The project predicts general categories. It does not read speed-limit numbers, determine traffic-light state, or make driving decisions. Traffic lights are included as related road signals. The current implementation is an educational prototype and has not been validated as a safety system.

The experimental scope is deliberately small. Generated images make the workflow quick to reproduce and provide known labels and coordinates. They also create an important limitation: success on simple drawings cannot demonstrate reliable recognition of real signs under changing weather, lighting, distance, or camera angle.

2. Methodology

2.1 Data and annotations

The generator draws a simple road background and a sign-like object, with variation in position, size, and background colour. Each annotation records `image_path`, `xmin`, `ymin`, `xmax`, `ymax`, and `sign_label`. The coordinates identify the object's rectangle. The preparation script checks required columns and positive box dimensions before dividing the rows into training, validation, and test groups.

The original pipeline generates ten images, two per category, and splits them into seven training examples, one validation example, and two test examples. The separate demonstration generates 120 images, 24 per category, using generator seed 123. Its split contains 84 training, 18 validation, and 18 test examples. All images come from the same generator, so the second split tests variations of simple drawings rather than a different real-world domain.

2.2 Preprocessing and classification

Training uses crops from the annotation rectangles rather than YOLO predictions. Each crop is resized to 224 by 224 pixels, converted to a tensor, and normalised. Augmentation applies horizontal flips, colour jitter, and rotations up to eight degrees. Evaluation uses fixed resizing and normalisation. Horizontal flips require review before applying this method to directional signs or signs containing text.

ResNet-18 extracts features from a crop. A linear layer produces five category scores, and the highest score supplies the prediction. Cross-entropy measures training error, and AdamW updates the model with learning rate 0.0003 and weight decay 0.0001. The demonstration trains from scratch for 15 epochs with batch size 16 and training seed 42. Pretrained classifier weights are supported but were disabled in both recorded experiments. The model has no unknown-category rejection.

2.3 Detection and video processing

Video inference uses pretrained `yolo11n.pt` weights. The current filter accepts standard COCO class IDs 9 and 11, corresponding to traffic light and stop sign. Confidence and IoU thresholds are 0.35 and 0.5. The detector has not been fine-tuned for the five project categories. Accepted rectangles are cropped and passed to the classifier. The program draws the predicted label and rectangle, then writes the frame to a video. The demo uses a PyTorch checkpoint, not ONNX inference.

3. Experiments and Results

3.1 Original workflow experiment

The DVC pipeline prepares ten synthetic images, trains for one epoch without pretrained classifier weights, and exports the classifier to ONNX. Final test evaluation is disabled for that command. The log records validation accuracy and F1 values of 1.0, but validation contains only one traffic-light image. A separate inspection found that this checkpoint predicted traffic light for all ten annotated crops. The validation result therefore cannot support a useful general accuracy claim.

3.2 Separate classifier and video experiment

A separate checkpoint was trained on the larger generated dataset without replacing the original artifacts. Direct evaluation of its best checkpoint correctly classified all 18 held-out annotated crops, giving crop accuracy 18/18. This result is specific to generated crops and is separate from detection quality or complete video recognition. The stored per-image predictions provide evidence for the count.

The video contains five held-out synthetic images, each repeated for four seconds, at ten frames per second. The following observations describe the processed examples.

Example	Detector result	Classifier result
Stop	Stop sign	Stop
Speed limit	No accepted detection	No crop
Traffic light	Traffic light	Traffic light
Pedestrian crossing	No accepted detection	No crop
No entry	Stop sign	No entry

The no-entry example illustrates a detector category error that the classifier corrects on this particular crop. It does not establish reliable no-entry detection. The missed examples show that correct annotated-crop predictions cannot guarantee recognition in the complete pipeline.

3.3 Timing and verification

Processing the 200-frame input took 9.91 seconds including model loading, approximately 20.18 frames per second. The recorded platform was macOS 26.6.2 on arm64, using CPU and two PyTorch threads. The input duration was 20 seconds. Every frame of the input and output videos was decoded to check frame count and dimensions. This timing concerns repeated generated scenes and does not establish real-road throughput, camera latency, or detector mAP.

4. Discussion and Reproducibility

The first experiment highlights the difficulty of interpreting a score from an extremely small validation set. A model predicting one category can succeed on a single matching example while failing on the other categories. Increasing the synthetic dataset and training duration produced correct held-out crop predictions, but both changes occurred together. This experiment does not isolate the individual effect of dataset size or epoch count.

The main remaining problem is detector coverage. Speed-limit and pedestrian-crossing examples never reach the classifier. The no-entry example passes the filter through an incorrect stop-sign detection. A detector trained for the intended categories, with a matching class map, is needed before assessing the complete task.

Git manages source changes. Hydra loads task settings, Lightning organises training, TensorBoard records training values, and DVC defines the original data, training, and export pipeline. A Dockerfile describes the environment, but a verified Docker run is not established here. The original classifier ONNX export exists; the separately trained video classifier is not that export. The larger experiment's data and weights remain local outputs outside the original DVC stages. The tracked demo video and results record support inspection, but do not provide those weights to a fresh clone.

The exact required template fork, representative real-data evaluation, detector mAP, and optimisation comparisons remain unverified. Future experiments should use appropriately licensed real images, keep related video frames together during splitting, train the detector, inspect category-level errors, and benchmark full-pipeline speed. ONNX speed comparisons require an implemented ONNX inference path and controlled measurements.

5. Conclusion

The prototype demonstrates data preparation, classifier training, model export, and annotated video generation. The separate synthetic experiment provides inspectable successes and missed detections. The main conclusion is that classification and detection must be evaluated separately and together. Real-data training and evaluation remain necessary before claiming reliable road-sign recognition or practical driver-assistance performance.

Evidence Sources

The report draws on `dvc.yaml`, the source and configuration files, the original training log, and the separate experiment record in `report/experiment_results.json`. The tracked output is `demo/traffic_sign_demo.mp4`. Proposed future experiments are not reported as completed results.